Van Concept naar Code: Een Diepgaande Gids voor.NET MAUI Architectuur

Deel I: De Fundamenten – Een Conceptuele Introductie tot.NET MAUI

Voordat de eerste regel code wordt geschreven, is het essentieel om een solide conceptueel fundament te leggen. Het begrijpen van de filosofie achter een framework is de sleutel tot het effectief gebruiken ervan. Dit deel introduceert.NET MAUI, legt de twee fundamentele programmeerparadigma's uit die de kern vormen van moderne UI-ontwikkeling, en identificeert de belangrijkste bouwstenen van een MAUI-applicatie.

1.1 Wat is.NET MAUI?

.NET Multi-platform App UI (MAUI) is de evolutie van Xamarin.Forms, een open-source framework van Microsoft voor het bouwen van native mobiele en desktop-applicaties vanuit één gedeelde C#-codebase. De kernpropositie is krachtig en efficiënt: schrijf de logica en de gebruikersinterface (UI) van een applicatie één keer, en implementeer deze vervolgens als een volledig native applicatie op Android, iOS, macOS en Windows.

MAUI lost een fundamenteel probleem op in softwareontwikkeling: de noodzaak om voor elk platform een aparte applicatie te ontwikkelen met platform-specifieke talen (zoals Kotlin/Java voor Android, Swift/Objective-C for iOS) en UI-frameworks. Door een abstractielaag te bieden, stelt MAUI ontwikkelaars in staat om de onderliggende platform-API's aan te spreken vanuit een gedeelde C# en XAML-omgeving. Het resultaat is geen web-app in een hulsje, maar een echte native applicatie die de prestaties en het uiterlijk levert die gebruikers van hun apparaten verwachten.

1.2 De Twee Filosofieën: Imperatief vs. Declaratief

Een van de meest voorkomende bronnen van verwarring voor beginnende UI-ontwikkelaars is de onuitgesproken aanwezigheid van twee verschillende manieren van denken over het bouwen van een interface. Het begrijpen van dit onderscheid is de sleutel tot het doorgronden van de rest van deze gids.

De Imperatieve Benadering: De "Hoe"-Methode

De imperatieve benadering is de meest traditionele en intuïtieve manier van programmeren. De ontwikkelaar geeft de computer een reeks expliciete, stapsgewijze instructies om een gewenst resultaat te bereiken. Het is een recept dat de computer moet volgen.

In de context van UI-ontwikkeling betekent dit dat de code de UI-elementen direct manipuleert. Denk aan instructies als:

1. "Zoek de knop met de naam 'AddButton'."
2. "Lees de tekst uit het invoerveld 'NewItemEntry'."
3. "Voeg deze tekst toe aan de lijst 'ItemsList'."
4. "Maak het invoerveld 'NewItemEntry' leeg."

Deze aanpak, vaak geassocieerd met "code-behind" bestanden (zoals MainPage.xaml.cs), geeft de ontwikkelaar volledige controle over elke stap van het proces. De logica is direct gekoppeld aan de UI-gebeurtenissen, zoals een klik op een knop.

De Declaratieve Benadering: De "Wat"-Methode

De declaratieve benadering is een modernere en abstractere manier van denken. In plaats van de computer te vertellen *hoe* iets moet worden gedaan, beschrijft de ontwikkelaar de *gewenste eindstaat*. Het framework is vervolgens verantwoordelijk voor het uitzoeken van de stappen die nodig zijn om die staat te bereiken en te behouden.

In de context van UI-ontwikkeling betekent dit dat de UI wordt beschreven als een functie van de applicatiedata (de "state"). Denk aan beschrijvingen als:

- "De tekst van dit label *is* altijd gelijk aan de waarde van de UserName eigenschap."
- "De lijst met items in deze weergave *is* een reflectie van de ToDoItems collectie."

- "De 'Verzenden' knop *is* ingeschakeld als en alleen als het IsFormValid veld true is."

De ontwikkelaar definieert deze relaties, en het framework zorgt ervoor dat de UI automatisch wordt bijgewerkt wanneer de onderliggende data verandert. Deze filosofie is de basis van XAML en het concept van "Data Binding".

Het door u aangeleverde cursusmateriaal mengt deze twee benaderingen zonder het onderscheid te maken, wat leidt tot verwarring. Deze gids zal ze eerst volledig scheiden om helderheid te scheppen, en vervolgens laten zien hoe de declaratieve aanpak de professionele standaard is voor het bouwen van robuuste en onderhoudbare applicaties.

1.3 De Sleutelcomponenten

Om deze filosofieën in de praktijk te brengen, maakt.NET MAUI gebruik van drie kerncomponenten:

- **XAML (eXtensible Application Markup Language):** Dit is een op XML-gebaseerde taal die wordt gebruikt om de UI van een applicatie op een declaratieve manier te beschrijven. In XAML definieert men de structuur van een pagina, de layout van de elementen, en de relaties tussen die elementen en de onderliggende data. Het is de taal van het "wat".
- **C# (C-Sharp):** Dit is de programmeertaal die wordt gebruikt voor alle applicatielogica. Het is de taal van het "hoe". In C# definieert men de datastructuren (de Models), de bedrijfslogica en de interacties (de ViewModels).
- **Data Binding:** Dit is het mechanisme dat de brug slaat tussen de declaratieve XAML-wereld en de C#-logica. Het is de "magische lijm" die ervoor zorgt dat de UI automatisch reageert op veranderingen in de data, en vice versa. Data Binding is de motor achter de declaratieve benadering in.NET MAUI.

Met dit conceptuele raamwerk op zijn plaats, kunnen we nu de structuur van een daadwerkelijk.NET MAUI-project verkennen.

Deel II: Anatomie van een.NET MAUI Project – Een Rondleiding door de Mappenstructuur

Wanneer een nieuw.NET MAUI-project wordt aangemaakt, presenteert Visual Studio een reeks bestanden en mappen. Deze structuur kan in het begin overweldigend lijken, maar elk onderdeel heeft een specifieke en logische rol. Een goed begrip van deze structuur is essentieel voor het navigeren en bouwen van een applicatie.

2.1 Het Startpunt: MauiProgram.cs

Dit bestand is de "ontstekingsmotor" van de applicatie. Het is het allereerste stuk code dat wordt uitgevoerd wanneer de app start. De primaire verantwoordelijkheid van MauiProgram.cs is het configureren en "bootstrappen" van de applicatie. Hier worden essentiële taken uitgevoerd, zoals:

- Het initialiseren van het MAUI-framework.
- Het registreren van lettertypen en andere resources.
- Het configureren van "services" voor Dependency Injection, een geavanceerd maar cruciaal concept voor het bouwen van schaalbare applicaties. Dit omvat het registreren van Views en ViewModels, zodat het framework weet hoe ze moeten worden aangemaakt en aan elkaar gekoppeld.

2.2 De Applicatieschil: App.xaml en App.xaml.cs

Nadat MauiProgram.cs zijn werk heeft gedaan, wordt de controle overgedragen aan de App klasse. Deze klasse vertegenwoordigt de applicatie als geheel en fungeert als de wortel van de UI-boom.

- **App.xaml:** Dit XAML-bestand wordt vaak gebruikt om applicatie-brede resources te definiëren, zoals kleuren, stijlen en templates die in de hele app consistent moeten zijn.
- **App.xaml.cs:** In de code-behind van dit bestand wordt het hoofdvenster of de eerste pagina van de applicatie ingesteld. Standaard wordt hier een AppShell of direct een MainPage geladen, wat de gebruiker te zien krijgt bij het opstarten.

2.3 De Platformen: De Platforms Map

Deze map bevat submappen voor elk ondersteund platform (Android, iOS, macOS, Windows). Hier bevindt zich de platform-specifieke "bootstrap" code en configuratie. Dit is de plek waar de gedeelde.NET MAUI-code wordt aangesloten op het native besturingssysteem. Voor 95% van het ontwikkelwerk hoeft men deze map niet aan te raken. MAUI abstraheert de complexiteit weg. Het is echter goed om te weten dat deze map bestaat voor de zeldzame gevallen waarin platform-specifieke aanpassingen of API-aanroepen nodig zijn.

2.4 De Schatkist: De Resources Map

Zoals de naam al doet vermoeden, is dit de centrale opslagplaats voor alle gedeelde assets van de applicatie. Dit omvat:

- **AppIcon:** De iconen voor de applicatie in verschillende formaten.
- **Images:** Afbeeldingen en iconen die in de UI worden gebruikt. MAUI kan SVG-bestanden (Scalable Vector Graphics) automatisch omzetten naar de juiste formaten voor elk platform.
- **Fonts:** Aangepaste lettertypebestanden.
- **Styles:** Een Styles.xaml bestand waar men herbruikbare stijlen kan definiëren om een consistent uiterlijk in de hele app te garanderen.
- **Raw:** Ruwe data-assets, zoals tekstbestanden of databases.

2.5 Uw Werkgebied: Views en Logica

De kern van het ontwikkelwerk vindt plaats in bestanden die de schermen (Views) van de applicatie definiëren. Standaard wordt een project geleverd met:

- **MainPage.xaml:** Dit is het XAML-bestand dat de visuele structuur van de hoofdpagina van de applicatie definieert. Hier plaatst men knoppen, labels, invoervelden en andere UI-elementen.
- **MainPage.xaml.cs:** Dit is het C# "code-behind" bestand dat direct is gekoppeld aan MainPage.xaml. In de traditionele, imperatieve benadering wordt hier de logica geschreven die reageert op gebruikersinteracties, zoals het afhandelen van een Clicked event van een knop.

De volgende tabel biedt een beknopt overzicht van de projectstructuur voor snelle referentie.

Bestand/Map	**Functie**	**Belangrijkste Rol in de Applicatie**
MauiProgram.cs	Configuratie & Opstarten	Initialiseert de app, registreert services en lettertypen. Het absolute startpunt.
App.xaml / .cs	Applicatieschil	Definieert globale stijlen en laadt het eerste venster/de eerste pagina van de app.
Platforms/	Platform-specifiek	Bevat de "lijmcode" die de gedeelde app verbindt met elk native platform.
Resources/	Gedeelde Assets	Centrale opslag voor afbeeldingen, lettertypen, iconen en globale stijlen.
MainPage.xaml	View Definitie	Beschrijft de layout en de elementen van een specifieke pagina met XAML.
MainPage.xaml.cs	View Logica (Code-Behind)	Bevat de C#-code die direct is gekoppeld aan de MainPage.xaml view.

Met dit begrip van de projectstructuur zijn we klaar om onze handen vuil te maken en een eerste applicatie te bouwen, waarbij we de concepten uit Deel I in de praktijk brengen.

Deel III: De Praktijk – Uw Eerste ToDo-App (De Code-Behind Manier)

Om de verschillen tussen de imperatieve en declaratieve benaderingen te illustreren, bouwen we een eenvoudige ToDo-applicatie op twee manieren. We beginnen met de meest directe methode: de code-behind aanpak. Deze aanpak is volledig imperatief. We zullen de UI-elementen expliciete namen geven en ze vanuit de C#-code direct aansturen.

Deze methode is nuttig om de basisprincipes te leren, maar zoals we later zullen zien, heeft het aanzienlijke nadelen voor grotere, complexere applicaties. Door deze aanpak eerst te isoleren en te perfectioneren, creëren we een helder startpunt en een duidelijke motivatie voor de overstap naar een robuustere architectuur.

3.1 Stap 1: De UI Definiëren in XAML

De eerste stap is het opzetten van de gebruikersinterface. We openen het MainPage.xaml bestand en definiëren de visuele elementen die nodig zijn voor onze ToDo-app.

De UI zal bestaan uit:

1. Een titel (Label).
2. Een invoerveld (Entry) voor nieuwe ToDo-items.
3. Een knop (Button) om een nieuw item toe te voegen.
4. Een lijst (CollectionView) om de bestaande ToDo-items weer te geven.

Cruciaal in deze fase is het gebruik van het x:Name attribuut. Dit attribuut geeft een UI-element een unieke naam, waardoor het vanuit het MainPage.xaml.cs code-behind bestand direct kan worden aangesproken. We laten in deze fase bewust alle {Binding} syntax weg om de twee paradigma's strikt gescheiden te houden.

MainPage.xaml (Code-Behind Versie):

XML

```
<?xml version="1.0" encoding="utf-8"?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
```

```
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        x:Class="MauiToDoApp.MainPage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <Label Text="Mijn ToDo Lijst" FontSize="Large" FontAttributes="Bold" />

        <Entry x:Name="NewToDoInput" Placeholder="Voeg een nieuwe taak toe..." />

        <Button x:Name="AddButton" Text="Toevoegen" Clicked="OnAddToDoClicked" />

        <CollectionView x:Name="ItemsView">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Label Text="{Binding.}" Padding="10" />
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>

</ContentPage>
```

Analyse van de XAML-code:

- **VerticalStackLayout:** Dit is een layout-container die zijn kinderen (de Label, Entry, etc.) verticaal onder elkaar plaatst.
- **x:Name="NewToDoInput":** We geven het Entry veld de naam NewToDoInput. Dit is onze "haak" om de ingevoerde tekst later in C# te kunnen lezen.
- **x:Name="AddButton":** De knop krijgt een naam, hoewel we deze in dit specifieke geval niet direct nodig hebben in de code.
- **Clicked="OnAddToDoClicked":** Dit is een event-abonnement. We instrueren de knop dat wanneer erop geklikt wordt, de methode met de naam OnAddToDoClicked in het MainPage.xaml.cs bestand moet worden uitgevoerd. Dit is een klassiek voorbeeld van imperatieve koppeling.
- **x:Name="ItemsView":** De CollectionView krijgt de naam ItemsView. Dit is essentieel, want we zullen deze lijst vanuit de C#-code moeten vullen met data.

3.2 Stap 2: De Data Voorbereiden in C#

Nu de UI is gedefinieerd, gaan we naar het MainPage.xaml.cs bestand om de data en de logica voor te bereiden. Hier definiëren we de lijst die onze ToDo-items zal bevatten.

Een veelgemaakte fout is om hiervoor een standaard List<string> te gebruiken. Dit werkt niet zoals verwacht in een UI-context. Wanneer een item aan een List<T> wordt toegevoegd, heeft de UI geen manier om te weten dat er een verandering heeft plaatsgevonden. De lijst op het scherm zal niet automatisch worden bijgewerkt.

De oplossing is het gebruik van ObservableCollection<T>. Deze speciale collectie, die zich in de System.Collections.ObjectModel namespace bevindt, implementeert de INotifyCollectionChanged interface. Dit betekent dat het een CollectionChanged event uitzendt telkens wanneer een item wordt toegevoegd, verwijderd, verplaatst of de hele lijst wordt gewist. UI-elementen zoals de CollectionView kunnen zich op dit event abonneren en weten daardoor precies wanneer ze zichzelf moeten verversen. Het gebruik van ObservableCollection is dus fundamenteel voor dynamische data in.NET MAUI.

MainPage.xaml.cs (Data Voorbereiding):

C#

```
using System.Collections.ObjectModel;

namespace MauiToDoApp;

public partial class MainPage : ContentPage
{
    // De datacollectie voor onze ToDo-items.
    // Gebruik ObservableCollection zodat de UI automatisch updates ontvangt.
    public ObservableCollection<string> Items { get; set; }

    public MainPage()
    {
        InitializeComponent();

        // Initialiseer de collectie.
        Items = new ObservableCollection<string>();

        //... verdere logica volgt in de volgende stap...
    }

    //... de event handler komt hier...
}
```

3.3 Stap 3: De UI en Data Koppelen (Imperatief)

Met de UI (ItemsView) en de data (Items) beide gedefinieerd, moeten we ze nu aan elkaar koppelen. In de imperatieve aanpak doen we dit met een directe, handmatige toewijzing in de code. We vertellen de CollectionView expliciet welke databron het moet gebruiken.

De ItemsSource eigenschap van de CollectionView is de sleutel. Deze eigenschap verwacht een collectie (zoals onze ObservableCollection) en zal voor elk item in die collectie een visueel element genereren op basis van de ItemTemplate die we in XAML hebben gedefinieerd.

We voegen deze koppelingscode toe aan de constructor van MainPage, direct na het initialiseren van de Items collectie.

MainPage.xaml.cs (Koppeling):

C#

```
public partial class MainPage : ContentPage
{
    public ObservableCollection<string> Items { get; set; }

    public MainPage()
    {
        InitializeComponent();

        Items = new ObservableCollection<string>();

        // IMPERATIEVE KOPPELING:
        // We wijzen onze C# collectie 'Items' direct toe aan de
        // ItemsSource eigenschap van het XAML element 'ItemsView'.
        this.ItemsView.ItemsSource = Items;
    }

    //... event handler...
}
```

Deze ene regel, this.ItemsView.ItemsSource = Items;, is de essentie van de imperatieve

datakoppeling. Er is geen magie; het is een directe opdracht van de code aan de UI.

3.4 Stap 4: De Interactie Implementeren

De laatste stap is het implementeren van de logica die wordt uitgevoerd wanneer de gebruiker op de "Toevoegen" knop klikt. In onze XAML hebben we al gespecificeerd dat de OnAddToDoClicked methode moet worden aangeroepen. Nu moeten we die methode in MainPage.xaml.cs definiëren.

De logica is als volgt:

1. Controleer of het invoerveld (NewToDoInput) niet leeg is.
2. Lees de tekst uit de Text eigenschap van het NewToDoInput element.
3. Voeg deze tekst toe aan onze Items ObservableCollection.
4. Maak het NewToDoInput veld leeg voor de volgende invoer.

MainPage.xaml.cs (Volledige Code):

C#

```
using System.Collections.ObjectModel;

namespace MauiToDoApp;

public partial class MainPage : ContentPage
{
    public ObservableCollection<string> Items { get; set; }

    public MainPage()
    {
        InitializeComponent();
        Items = new ObservableCollection<string>();
        this.ItemsView.ItemsSource = Items;
    }

    private void OnAddToDoClicked(object sender, EventArgs e)
    {
        // Controleer of er tekst is ingevoerd.
```

```
        if (!string.IsNullOrWhiteSpace(NewToDoInput.Text))
        {
            // Voeg het nieuwe item toe aan de ObservableCollection.
            // De CollectionView zal zichzelf automatisch bijwerken.
            Items.Add(NewToDoInput.Text);

            // Maak het invoerveld leeg.
            NewToDoInput.Text = string.Empty;
        }
    }
}
```

Analyse van de OnAddToDoClicked methode:

- De methode heeft directe toegang tot NewToDoInput en Items omdat ze beide lid zijn van de MainPage klasse.
- NewToDoInput.Text leest de waarde rechtstreeks uit het UI-element.
- Items.Add(...) voegt de waarde toe aan de datacollectie. Omdat Items een ObservableCollection is en de ItemsSource van ItemsView hieraan is gekoppeld, wordt de UI automatisch bijgewerkt zonder dat we een extra commando hoeven te geven.
- NewToDoInput.Text = string.Empty; manipuleert de UI direct om het veld te wissen.

We hebben nu een volledig functionele ToDo-applicatie gebouwd met de pure code-behind, imperatieve methode. Elk stukje logica is direct gekoppeld aan de UI. In het volgende deel zullen we de nadelen van deze aanpak analyseren en de applicatie refactoren naar een veel krachtigere en professionelere architectuur.

Deel IV: De Grote Refactoring – Van Code-Behind naar de Kracht van MVVM

De code-behind aanpak die we in Deel III hebben geïmplementeerd, werkt prima voor kleine, eenvoudige applicaties. Echter, naarmate een applicatie groeit in complexiteit, begint deze aanpak aanzienlijke nadelen te vertonen. Dit deel introduceert de Model-View-ViewModel (MVVM) architectuur als oplossing voor deze problemen en leidt u stap voor stap door de transformatie van onze ToDo-app.

4.1 Waarom Refactoren? De Grenzen van Code-Behind

De applicatie uit Deel III heeft twee fundamentele zwaktes:

1. **Hoge Koppeling (Tight Coupling):** De UI (View, MainPage.xaml) en de logica (code-behind, MainPage.xaml.cs) zijn onlosmakelijk met elkaar verbonden. De C#-code is afhankelijk van de specifieke namen en types van de UI-elementen (NewToDoInput, ItemsView). Als een ontwerper besluit om de Entry te vervangen door een ander type invoer-control, moet de C#-code ook worden aangepast. De logica en de presentatie kunnen niet onafhankelijk van elkaar evolueren.
2. **Slechte Testbaarheid:** Het is extreem moeilijk, zo niet onmogelijk, om de logica in MainPage.xaml.cs te testen zonder de volledige UI te laden. Om de OnAddToDoClicked methode te testen, zou men een daadwerkelijke Entry en CollectionView moeten simuleren. Dit maakt het schrijven van geautomatiseerde unit tests, een hoeksteen van professionele softwareontwikkeling, zeer omslachtig.

MVVM is een architectonisch patroon dat is ontworpen om precies deze problemen op te lossen door een strikte scheiding van verantwoordelijkheden af te dwingen.

4.2 Introductie van de Actoren

MVVM splitst de applicatie op in drie verschillende, losjes gekoppelde componenten:

- **Model:** Vertegenwoordigt de data en de domeinlogica van de applicatie. Dit zijn simpele C#-klassen (vaak POCOs - Plain Old CLR Objects) die de data van de applicatie beschrijven. In onze ToDo-app zou dit een ToDoItem klasse kunnen zijn die de tekst van

een taak en een status (bijv. voltooid/niet voltooid) bevat.

- **View:** De gebruikersinterface (UI). In.NET MAUI is dit het .xaml bestand. De View is verantwoordelijk voor de presentatie van de data en het doorgeven van gebruikersinteracties (zoals klikken en typen) aan de ViewModel. De View bevat idealiter *geen* logica, behalve wat strikt noodzakelijk is voor de UI (zoals animaties).
- **ViewModel:** De "lijm" tussen het Model en de View. De ViewModel is een C#-klasse die de staat (data) en het gedrag (logica) van de View blootstelt. Het haalt data uit de Models en formatteert deze zodat de View ze gemakkelijk kan weergeven. Het stelt ook commando's (ICommand) bloot waaraan de View kan binden om acties uit te voeren. Cruciaal is dat de ViewModel *geen enkele kennis* heeft van de specifieke UI-elementen in de View. Het weet niet of data wordt weergegeven in een Label of een Button; het stelt simpelweg data en acties beschikbaar.

Laten we deze componenten nu creëren voor onze ToDo-app.

Model: ToDoItem.cs
Voor nu houden we het simpel en blijven we string gebruiken. In een echte applicatie zou dit een klasse zijn:

C#

```
// Voorbeeld van een Model-klasse
public class ToDoItem
{
    public string TaskName { get; set; }
    public bool IsCompleted { get; set; }
}
```

Voor de rest van deze gids blijven we de ObservableCollection<string> gebruiken om de focus op het MVVM-patroon te houden.

ViewModel: ToDoListViewModel.cs
Dit is een nieuwe C#-klasse die het brein van onze pagina wordt. We verplaatsen alle data en logica van MainPage.xaml.cs naar deze nieuwe klasse.

4.3 De Sleutelsteen: Het BindingContext Concept Ontrafeld

Hoe weet de View (de XAML) welke ViewModel het moet gebruiken als databron? Het antwoord is de BindingContext.

Elk visueel element in.NET MAUI heeft een BindingContext eigenschap. Dit is een object dat fungeert als de standaard databron voor alle data-bindings binnen dat element en zijn kinderen. Wanneer we de BindingContext van een hele pagina instellen op een instantie van onze ToDoListViewModel, zeggen we in feite: "Voor alle {Binding}-expressies op deze pagina, zoek de eigenschappen op dit ToDoListViewModel object."

De meest eenvoudige manier om dit in te stellen is in de constructor van de code-behind van de View.

MainPage.xaml.cs (Na de refactoring):

C#

```
namespace MauiToDoApp;

public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();

        // Stel de BindingContext in op een nieuwe instantie van onze ViewModel.
        // Dit is de enige regel code die overblijft in de code-behind!
        BindingContext = new ToDoListViewModel();
    }
}
```

Met deze ene regel is de brug geslagen. De MainPage.xaml is nu verbonden met de ToDoListViewModel. De code-behind is nu bijna leeg, precies zoals het hoort in een MVVM-architectuur.

4.4 De Magie van Notificaties: INotifyPropertyChanged Demystificeren

Een probleem blijft: als een eigenschap in de ViewModel verandert, hoe weet de View dan dat het zichzelf moet bijwerken? Dit is waar de INotifyPropertyChanged interface om de hoek komt kijken.

Deze interface, die zich in System.ComponentModel bevindt, definieert slechts één event: PropertyChanged. Een klasse die deze interface implementeert, kan dit event aanroepen om te signaleren dat de waarde van een van zijn eigenschappen is veranderd. De data-binding-engine van MAUI abonneert zich automatisch op dit event.

Dit is een implementatie van het *Observer Pattern*. De View "observeert" de ViewModel. Wanneer de ViewModel een verandering aankondigt via het PropertyChanged event, reageert de View door het betreffende UI-element bij te werken.

Laten we dit implementeren in onze ToDoListViewModel. We verplaatsen de Items collectie en voegen een nieuwe eigenschap NewToDoText toe om de tekst uit het Entry veld bij te houden.

ToDoListViewModel.cs (met INotifyPropertyChanged):

C#

```
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace MauiToDoApp.ViewModels;

public class ToDoListViewModel : INotifyPropertyChanged
{
    // 1. Implementeer de interface
    public event PropertyChangedEventHandler PropertyChanged;

    // 2. De datacollectie
    public ObservableCollection<string> Items { get; set; }

    private string _newToDoText;
    // 3. De eigenschap voor het invoerveld
```

```
    public string NewToDoText
    {
        get => _newToDoText;
        set
        {
            if (_newToDoText!= value)
            {
                _newToDoText = value;
                // Roep de notificatie aan!
                OnPropertyChanged();
            }
        }
    }

    public ToDoListViewModel()
    {
        Items = new ObservableCollection<string>();
    }

    // 4. De helper-methode om het event aan te roepen
    protected void OnPropertyChanged([CallerMemberName] string propertyName = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}
```

Analyse van de code:

1. De klasse implementeert INotifyPropertyChanged.
2. De Items collectie is hier nu de eigenaar van.
3. De NewToDoText eigenschap heeft een private "backing field" _newToDoText. In de set accessor, nadat de waarde is bijgewerkt, roepen we OnPropertyChanged() aan. Dit is de cruciale stap die het signaal naar de View stuurt.
4. De OnPropertyChanged methode is een standaard helper. Het [CallerMemberName] attribuut vult automatisch de naam van de aanroepende eigenschap in ("NewToDoText" in dit geval), wat typefouten voorkomt.

4.5 Acties Ontkoppelen: De Kracht van ICommand

We hebben de data ontkoppeld, maar hoe zit het met de acties, zoals het klikken op de knop? In de code-behind gebruikten we een Clicked event handler. Dit creëert een directe koppeling tussen de View en de code.

De MVVM-manier om acties te hanteren is met ICommand. ICommand is een interface die een actie inkapselt. Het is een object dat twee dingen bevat:

1. Een Execute methode: de code die moet worden uitgevoerd.
2. Een CanExecute methode: een functie die bepaalt of de actie momenteel mag worden uitgevoerd (bijvoorbeeld, de "Toevoegen" knop uitschakelen als er geen tekst is ingevoerd).

Dit is een implementatie van het *Command Pattern*. De Button in de View weet niets meer over de specifieke C#-methode die het aanroept. Het weet alleen dat het gebonden is aan een ICommand object en dat het de Execute methode van dat object moet aanroepen wanneer erop wordt geklikt. Dit ontkoppelt de "trigger" (de knop) volledig van de "actie" (de logica in de ViewModel).

We voegen een AddToDoCommand toe aan onze ViewModel.

ToDoListViewModel.cs (met ICommand):

C#

```
//... (using statements en INotifyPropertyChanged implementatie)...
using System.Windows.Input; // Namespace voor ICommand

public class ToDoListViewModel : INotifyPropertyChanged
{
    //... (Properties: Items, NewToDoText)...

    // De Command eigenschap
    public ICommand AddToDoCommand { get; }

    public ToDoListViewModel()
    {
        Items = new ObservableCollection<string>();
        // Initialiseer de Command in de constructor
```

```
        AddToDoCommand = new Command(AddToDo);
    }

    // De methode die de daadwerkelijke logica bevat
    private void AddToDo()
    {
        if (!string.IsNullOrWhiteSpace(NewToDoText))
        {
            Items.Add(NewToDoText);
            NewToDoText = string.Empty; // Dit triggert OnPropertyChanged voor NewToDoText!
        }
    }

    //... (OnPropertyChanged methode)...
}
```

Analyse van de code:

- We voegen een public ICommand AddToDoCommand eigenschap toe.
- In de constructor maken we een nieuwe Command aan. De Command klasse is een standaardimplementatie van ICommand in MAUI. We geven de AddToDo methode mee als de actie die moet worden uitgevoerd.
- De AddToDo methode bevat nu de logica die voorheen in OnAddToDoClicked stond. Merk op dat het nu werkt met de NewToDoText eigenschap, niet met een direct UI-element. Wanneer we NewToDoText = string.Empty; instellen, wordt de set accessor van de eigenschap aangeroepen, die op zijn beurt OnPropertyChanged() triggert. De data-binding engine ziet dit en maakt automatisch het Entry veld in de UI leeg.

4.6 De Beloning: Een Volledig Declaratieve View

Nu alle data (Items, NewToDoText) en alle logica (AddToDoCommand) zich in de ToDoListViewModel bevinden, kunnen we de MainPage.xaml herschrijven naar een volledig declaratieve vorm. We vervangen alle x:Name attributen en event handlers door {Binding} expressies.

MainPage.xaml (MVVM Versie):

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="MauiToDoApp.MainPage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <Label Text="Mijn ToDo Lijst (MVVM)" FontSize="Large" FontAttributes="Bold" />

        <Entry Placeholder="Voeg een nieuwe taak toe..."
               Text="{Binding NewToDoText}" />

        <Button Text="Toevoegen"
                Command="{Binding AddToDoCommand}" />

        <CollectionView ItemsSource="{Binding Items}">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Label Text="{Binding.}" Padding="10" />
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>

</ContentPage>
```

Analyse van de bindings:

- Text="{Binding NewToDoText}": De Text van de Entry is nu direct gekoppeld aan de NewToDoText eigenschap in de ViewModel. Standaard is dit een TwoWay binding.

Wanneer de gebruiker typt, wordt de NewToDoText eigenschap in de ViewModel bijgewerkt. Wanneer de code NewToDoText verandert, wordt de tekst in de Entry bijgewerkt.
- Command="{Binding AddToDoCommand}": De klik-actie van de Button is gekoppeld aan de AddToDoCommand. De knop roept nu de Execute methode van dit commando aan, zonder te weten wat die methode doet.
- ItemsSource="{Binding Items}": De CollectionView is gekoppeld aan de Items ObservableCollection in de ViewModel.

De cirkel is rond. De View is nu een pure, declaratieve beschrijving van de UI, volledig gestuurd door de staat van de ViewModel. De code-behind is leeg, en de ViewModel bevat alle logica, volledig onafhankelijk van de UI.

De volgende tabel zet de twee benaderingen direct naast elkaar om de voordelen van MVVM te verduidelijken.

Aspect	**Code-Behind Aanpak (Deel III)**	**MVVM Aanpak (Deel IV)**	**Voordelen van MVVM**
Locatie van Data (State)	In het code-behind bestand (MainPage.xaml.cs) als public ObservableCollection<string> Items.	In de ViewModel (ToDoListViewModel.cs) als public ObservableCollection<string> Items.	Data is gescheiden van de UI-implementatie, kan hergebruikt worden.
Locatie van Logica	In de event handler OnAddToDoClicked in MainPage.xaml.cs.	In de AddToDo methode, aangeroepen door AddToDoCommand in ToDoListViewModel.cs.	Logica is onafhankelijk van de UI, waardoor het makkelijker te testen en te onderhouden is.
Koppeling UI & Logica	**Hoog (Tight).** Code-behind manipuleert direct UI-elementen via x:Name (bv. NewToDoInput.Text).	**Laag (Loose).** View en ViewModel communiceren via data binding en commands. ViewModel heeft geen kennis van de	Wijzigingen in de UI (bv. Entry vervangen door Editor) vereisen geen wijzigingen in de ViewModel.

		View.	
Testbaarheid	**Zeer moeilijk.** Vereist een UI-instantie om de logica in de event handler te testen.	**Eenvoudig.** De ToDoListViewModel kan als een gewone C#-klasse worden geïnstantieerd en getest in een unit test project, zonder UI.	Verhoogt de betrouwbaarheid en kwaliteit van de software aanzienlijk.

Deel V: De Cirkel is Rond – De "Waarom"-Vraag Beantwoord

We hebben nu dezelfde applicatie op twee manieren gebouwd. De MVVM-versie vereiste meer initiële opzet: het aanmaken van een ViewModel, het implementeren van INotifyPropertyChanged, en het opzetten van ICommand. De vraag is: wat is de uiteindelijke winst? Dit deel verbindt de abstracte voordelen van MVVM direct met de code die we hebben geschreven.

- **Scheiding van Verantwoordelijkheden (Separation of Concerns):** Dit is het meest fundamentele voordeel. In onze MVVM-app heeft elk onderdeel een duidelijke, enkele taak.
  - De **View** (MainPage.xaml) is alleen verantwoordelijk voor de presentatie. Het is "dom" en weet alleen hoe het data moet weergeven en interacties moet doorgeven.
  - De **ViewModel** (ToDoListViewModel.cs) is verantwoordelijk voor de staat en het gedrag van de View. Het bevat alle logica en data die de View nodig heeft.
  - Het Model (in ons geval de string, maar idealiter een ToDoItem klasse) is alleen verantwoordelijk voor het representeren van de domeindata.
    Deze scheiding maakt de code veel georganiseerder en gemakkelijker te begrijpen.
- **Testbaarheid (Testability):** Dit is misschien wel het meest concrete en krachtige voordeel. De ToDoListViewModel is een POCO (Plain Old CLR Object). Het heeft geen afhankelijkheden van het.NET MAUI UI-framework. Dit betekent dat men een unit test project kan opzetten en de ViewModel volledig kan testen. Men kan een test schrijven die:
  1. Een nieuwe ToDoListViewModel aanmaakt.
  2. De NewToDoText eigenschap instelt op "Leer MAUI".
  3. De AddToDoCommand.Execute() methode aanroept.
  4. Controleert of de Items collectie nu één item bevat en of de NewToDoText

eigenschap weer leeg is.
Dit alles kan worden gedaan zonder ooit een venster te tonen of een knop te simuleren. Dit is onmogelijk met de code-behind aanpak, waar de logica onlosmakelijk verbonden is met de UI-events.

- **Onderhoudbaarheid (Maintainability):** Stel dat het ontwerp van de applicatie verandert. De Entry en Button moeten worden vervangen door een nieuw, op maat gemaakt invoer-component met een geïntegreerde "plus"-knop.
  - In de **code-behind** aanpak zou dit waarschijnlijk vereisen dat de OnAddToDoClicked event handler en de manier waarop de tekst wordt gelezen, moeten worden aangepast.
  - In de **MVVM** aanpak hoeft men alleen de XAML in de View aan te passen. Zolang het nieuwe component bindt aan dezelfde NewToDoText en AddToDoCommand eigenschappen, hoeft er *geen enkele regel code* in de ToDoListViewModel te worden gewijzigd. De logica blijft intact, ongeacht hoe de presentatie verandert. Dit maakt het aanpassen en uitbreiden van de applicatie op de lange termijn veel eenvoudiger en minder foutgevoelig.
- **Samenwerking (Collaboration):** In een teamomgeving kunnen verschillende specialisten nu parallel werken. Een UI/UX-ontwerper kan zich volledig richten op het perfectioneren van de XAML-bestanden, terwijl een softwareontwikkelaar zich concentreert op de logica en dataverwerking in de ViewModels. Omdat de View en ViewModel alleen een "contract" delen (de namen van de properties en commands waaraan wordt gebonden), kunnen ze onafhankelijk van elkaar worden ontwikkeld.

De initiële investering in het opzetten van de MVVM-structuur betaalt zich vele malen terug in de vorm van een schonere, robuustere, beter testbare en gemakkelijker te onderhouden applicatie.

Deel VI: De Volgende Stap – Professionele Koppeling via Dependency Injection

We hebben een cruciale stap gezet door de BindingContext in de code-behind van de View in te stellen: BindingContext = new ToDoListViewModel();. Hoewel dit functioneel is en een uitstekende manier om het concept te leren, creëert het nog steeds een directe koppeling: de MainPage (View) is nu direct afhankelijk van de *concrete implementatie* van de ToDoListViewModel.

In grote, professionele applicaties is dit suboptimaal. Wat als de ToDoListViewModel zelf afhankelijkheden heeft, zoals een service om de ToDo-items op te slaan in een database? Moet de MainPage dan ook weten hoe die service moet worden aangemaakt? Dit leidt tot een

keten van afhankelijkheden die de code weer broos maakt.

De oplossing hiervoor is een patroon genaamd **Dependency Injection (DI)**.

De Oplossing: Dependency Injection (DI)

Het idee achter DI is simpel: in plaats van dat een object zijn eigen afhankelijkheden creëert (zoals new ToDoListViewModel()), worden die afhankelijkheden van buitenaf "geïnjecteerd", meestal via de constructor.

De View zegt niet langer: "Ik heb een *nieuwe ToDoListViewModel* nodig." In plaats daarvan zegt het: "Ik heb *een* ViewModel nodig die voldoet aan de eisen voor deze pagina." Een centraal "DI-container" systeem is vervolgens verantwoordelijk voor het aanmaken van de ViewModel (en al zijn sub-afhankelijkheden) en het doorgeven aan de View wanneer deze wordt aangemaakt.

In.NET MAUI is dit DI-systeem ingebouwd en wordt het geconfigureerd in MauiProgram.cs.

De Praktijk: Registreren en Injecteren

De professionele workflow bestaat uit twee stappen:

1. Registreer de View en ViewModel in MauiProgram.cs:
We vertellen de DI-container welke klassen het moet kennen en hoe ze moeten worden aangemaakt.

- AddTransient: Maakt elke keer een nieuwe instantie aan wanneer erom wordt gevraagd.
- AddSingleton: Maakt één enkele instantie aan en hergebruikt die voor de hele levensduur van de app.

C#

```
// In MauiProgram.cs

using MauiToDoApp.ViewModels; // Namespace van de ViewModel
```

```
using MauiToDoApp; // Namespace van de View

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
          .UseMauiApp<App>()
          .ConfigureFonts(fonts =>
           {
               fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
               fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
           });

        // Registreer de ViewModel en de View
        builder.Services.AddTransient<ToDoListViewModel>();
        builder.Services.AddTransient<MainPage>();

        return builder.Build();
    }
}
```

2. Injecteer de ViewModel in de View via de Constructor:
Nu de DI-container weet hoe een ToDoListViewModel moet worden gemaakt, kunnen we de MainPage aanpassen om deze in de constructor te vragen. De DI-container zal deze aanvraag automatisch afhandelen.
MainPage.xaml.cs (DI Versie):

C#

```
namespace MauiToDoApp;

public partial class MainPage : ContentPage
{
    // De View vraagt om de ViewModel in zijn constructor.
    public MainPage(ToDoListViewModel viewModel)
    {
        InitializeComponent();
```

```
        // De DI-container levert de 'viewModel' instantie.
        // We hoeven alleen nog de BindingContext in te stellen.
        BindingContext = viewModel;
    }
}
```

De regel BindingContext = new ToDoListViewModel(); is verdwenen. De MainPage creëert zijn ViewModel niet meer zelf. Het is nu volledig ontkoppeld. De MainPage weet alleen dat het een object van het type ToDoListViewModel nodig heeft, maar niet hoe dat object wordt geconstrueerd.

Deze aanpak is de gouden standaard voor het bouwen van schaalbare, onderhoudbare en testbare.NET MAUI-applicaties. Het voltooit de reis van een strak gekoppelde code-behind structuur naar een losjes gekoppelde, professionele architectuur die klaar is voor complexe, real-world scenario's.